Título:

Testes Automatizados para Diagnóstico de Sistema com API Externa

---

Descrição:

Você recebeu o sistema **AgroClima Inteligente**, que apresenta falhas.

Seu objetivo **NÃO é sair corrigindo direto**.

Primeiro você deve:

1. Criar **testes automatizados**
2. Usar os testes para **identificar os erros**
3. Corrigir o sistema até que todos os testes passem

Objetivo da atividade

- Aplicar testes unitários e de integração
- Identificar falhas via testes
- Corrigir código com base nos testes

PARTE PRINCIPAL (OBRIGATÓRIA)

1. Testes Unitários

Crie testes para:

📁 **irrigationRule.js**

Casos obrigatórios:

Temperatura 35 → Irrigação URGENTE
Temperatura 25 → Irrigação moderada
Temperatura 10 → Não irrigar

📁 **weatherService.js**

Você deve:

- Mockar a API (NÃO chamar API real)
- Validar parsing do JSON

---

2. Testes de Integração

Testar o endpoint:

GET /irrigation

Validar:

- Status HTTP
- Estrutura do JSON
- Dados retornados

REGRAS IMPORTANTES

- ❌ Não pode corrigir o código antes dos testes
- ✔ Testes devem falhar inicialmente
- ✔ Após correções, todos os testes devem passar

FERRAMENTAS

- Jest
- Supertest

ESTRUTURA DE TESTES ESPERADA

```
tests/
├── irrigationRule.test.js
├── weatherService.test.js
└── integration.test.js
```

EXEMPLOS (GUIA PARA OS ALUNOS)

📁 irrigationRule.test.js

```
const getAdvice = require("../backend/utils/irrigationRule");

test("temperatura alta", () => {
```

```
  expect(getAdvice(35)).toBe("Irrigação URGENTE");
});
```

📁 weatherService.test.js (com mock)

```
const axios = require("axios");
const getWeather = require("../backend/services/weatherService");

jest.mock("axios");

test("retorna temperatura corretamente", async () => {
  axios.get.mockResolvedValue({
    data: {
      current_weather: {
        temperature: 28
      }
    }
  });

  const result = await getWeather(-12, -38);
  expect(result.temperature).toBe(28);
});
```

📁 integration.test.js

```
const request = require("supertest");
const app = require("../backend/server");

jest.mock("../backend/services/weatherService", () => {
  return jest.fn(() => Promise.resolve({ temperature: 32 }));
});

test("retorna dados corretos", async () => {
  const res = await request(app)
    .get("/irrigation?lat=-12&lon=-38");

  expect(res.statusCode).toBe(200);
  expect(res.body.temperature).toBeDefined();
  expect(res.body.advice).toBeDefined();
});
```

COMO RODAR OS TESTES

Instalar dependências:

npm install jest supertest

Rodar testes:

npx jest

Para rodar o código

cd backend
npm install express axios cors
node server.js

Abrir no navegador:

http://localhost:3000